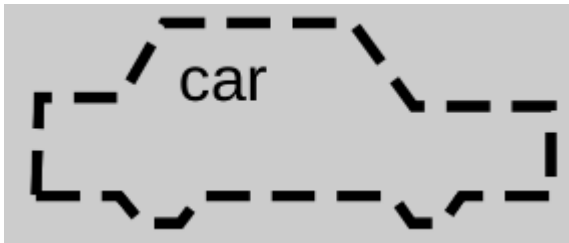


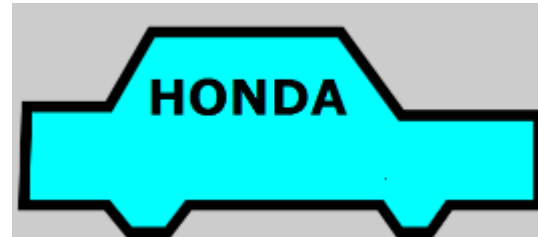
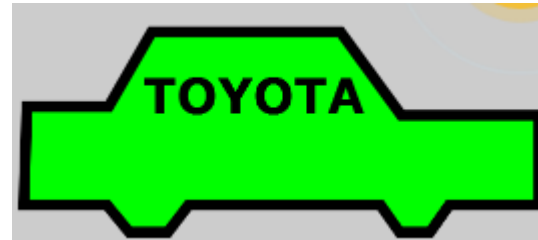
1. Phân loại các lớp đối tượng
2. Các thành phần trong một lớp đối tượng
3. Phương pháp xác định các lớp đối tượng
4. Xác định mối quan hệ giữa các lớp
5. Bài tập

1. Phân loại các lớp đối tượng

- Class:



- Object:



Phân biệt class và struct

- Struct trong C# dùng định nghĩa kiểu gồm nhiều trường dữ liệu, trong C# struct còn có thể chứa method và constructor.
- Quyền truy cập các thành viên trong struct là public, khác với quyền truy cập các thành viên trong class thường là private

2. Các thành phần trong một lớp đối tượng

- Thuộc tính: Các thuộc tính được khai báo giống như khai báo biến trong C
- Phương thức: Các phương thức được khai báo giống như khai báo hàm trong C. Có hai cách định nghĩa thi hành của một phương thức
 - Định nghĩa thi hành trong lớp
 - Định nghĩa thi hành ngoài lớp

3. Phương pháp xác định các lớp đối tượng

Đầu tiên ta nên tiếp cận yêu cầu theo hướng từ trên xuống , tức là từ mục đích quan trọng nhất sau đó tìm cách chia nhỏ mục đích ấy ra , ví dụ : tạo ứng dụng quản lí thông tin sinh viên , thì mục đích quan trọng nhất là quản lí được sinh viên, tuy nhiên mục đích này quá chung, ta tiếp tục phân tích ,thấy rằng quản lí sinh viên có thể gồm

- 1: quản lí điểm (xem điểm , cập nhật điểm , chỉnh sửa điểm ...)
- 2: quản lí về thông tin sinh viên (gồm học tên nơi ở , số điện thoại ,...)
- 3: tìm kiếm tra cứu thông tin sinh viên

3. Phương pháp xác định các lớp đối tượng

Đến bước này ta sẽ tìm một đối tượng (có thể hiểu nôm na là người) có thể dùng để nhóm các hành động trên ,

Ta để ý ở mục 2 , họ tên , nơi ở chính là các thuộc tính cơ bản của sinh viên vì vậy ta tạo ra một đối tượng “SinhVien” (class SinhVien) để lưu trữ và quản lí mục 2 .

Và ta cũng nhận ra các thao tác ở mục 1 và 3 là công việc của các thầy giáo vụ, vậy ta tạo ra một đối tượng “GiaoVu” với các phương thức xem điểm, cập nhật điểm , v.v

3. Phương pháp xác định các lớp đối tượng

Từ đây ta sẽ thấy yêu cầu đặt ra quy về tương tác giữa hai đối tượng là SinhVien và GiaoVu , lúc này ta sẽ dùng hai đối tượng này để tạo ra chương trình quản lí sinh viên đơn giản.

Chú ý khái niệm, lúc này ta chỉ có trong tay 2 đối tượng ta dùng 2 đối tượng này để giải quyết vấn đề chính là hướng tiếp cận từ dưới.

4. XÁC ĐỊNH MỐI QUAN HỆ GIỮA CÁC LỚP

- Là quan hệ giữa 2 phần tử trong mô hình mà thay đổi ở phần tử này (phần tử độc lập) có thể gây ra thay đổi ở phần tử kia (phần tử phụ thuộc).
- Là loại quan hệ giữa 2 object
- ClassA và ClassB không có quan hệ Association
 - Trong ClassA có sử dụng **biến toàn cục (kiểu B)**, hoặc sử dụng **phương thức/thuộc tính** static của ClassB
 - Ký hiệu : **A use-a B** , bằng mũi tên 1 chiều nét đứt , từ bên phụ thuộc sang bên độc lập.



- ClassA “phụ thuộc” vào ClassB ;
- Client → Supplier (phần tử phục thuộc → phần tử độc lập)
Dependency còn có một số biểu hiện khác , thường dùng các stereotype sau :

- **<<use>>** : chỉ rằng ngữ nghĩa của lớp gốc (mũi tên) phụ thuộc vào lớp ngọn (mũi tên). Đặc biệt trong trường hợp lớp gốc dùng lớp ngọn làm tham số trong 1 số method của nó
- **<<permit>>** : chỉ rằng lớp gốc được quyền truy cập 1 cách đặc biệt vào lớp ngọn (chẳng hạn truy cập các thao tác riêng tư). Tương ứng với khái niệm friend trong C++
- **<<refine>>** : chỉ rằng lớp gốc ở 1 mức độ tinh chế cao hơn từ lớp ngọn. Chẳng hạn 1 lớp độc lập ở giai đoạn thiết kế nhằm tinh chế cùng lớp đó lập ở giai đoạn phân tích

Lưu ý : Phân biệt giữa Dependency và Association

- Association là quan hệ cấu trúc
- Dependency là quan hệ phi cấu trúc

- Giữa 2 object của 2 lớp có sự ghép cặp (vợ – chồng , thầy – trò , khách hàng – hóa đơn ...) . Tập hợp các kết nối cùng loại (cùng ý nghĩa) giữa các object của 2 lớp tạo thành mối liên kết association , quan hệ giữa 2 tập hợp (2 lớp)
- Là mỗi liên hệ giữa 2 lớp có role, role là tên vai trò của mỗi liên kết : vd như : của , cho , có , liên kết tới , trao đổi với , (thường tên role có kèm theo 1 mũi tên để chỉ hướng quan hệ áp dụng từ lớp nào sang lớp nào)
- Ký hiệu : ***A has-a B***



Ý nghĩa : (trường hợp mũi tên không có chiều)

Hoặc : Trong ClassA có thuộc tính có kiểu là ClassB

Hoặc : Trong ClassB có thuộc tính có kiểu là ClassA

Nhận xét: Về mặt lập trình, thuộc tính có thể được lưu trữ dạng biến đơn, biến mảng, hay biến con trỏ

Có hoặc không có bản số cũng được

Nếu có mũi tên 1 chiều , chỉ ra chiều đối tượng thuộc lớp này chỉ có gọi đối tượng của lớp kia, không có chiều ngược lại

Nếu không có mũi tên nào thì tương đương là mũi tên 2 chiều , hoặc chiều không quan trọng.

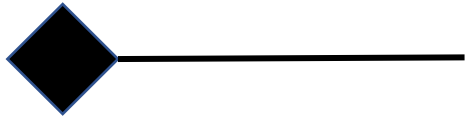
Multiplicity : , bản số , lượng số , số object bên này tham gia vào mỗi kết hợp so với 1 object bên kia

- Đã xác định được ClassA và ClassB có quan hệ Association với nhau
- Xác định rõ hơn:
- Trong object của ClassA có chứa (trong phần thuộc tính) object của ClassB
- ObjectX của ClassA bị hủy thì ObjectY của ClassB (bên trong ObjectX) vẫn có thể còn tồn tại
- Còn gọi là shared-aggregation. Một dạng của nối kết, trong đó một phần tử này chứa các phần tử khác.
- Ký hiệu :

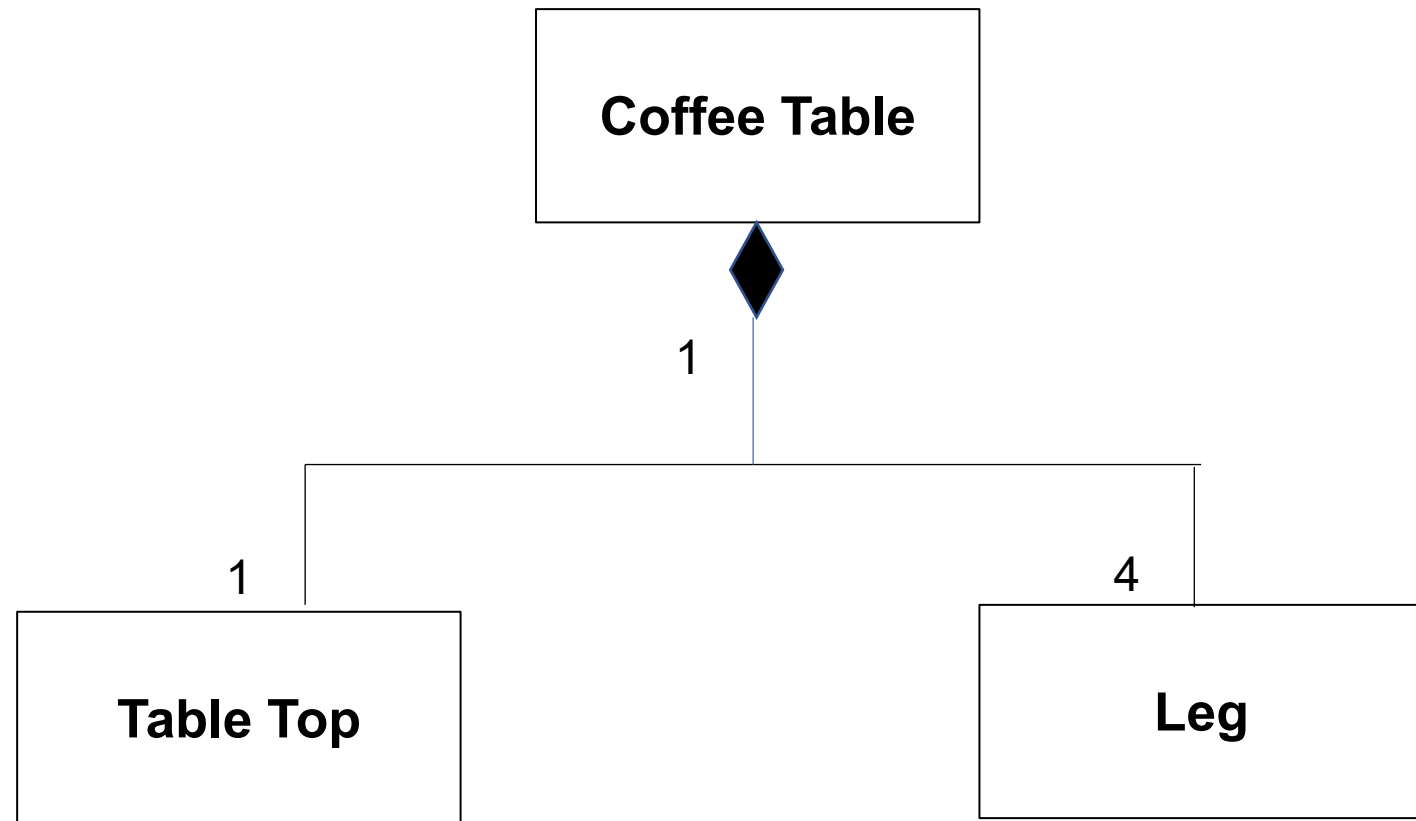


- Ý nghĩa : còn gọi là : **Whole A – Part B** . Nghĩa là A được tạo từ nhiều B kết hợp lại , và B có thể tạo ra độc lập , không cần phải tạo ra A , B có thể cùng thuộc 1 whole khác A.
- Chú ý : Từ **share** ở đây có nghĩa là , B có thể là bộ phận của **whole** khác, do đó A bị hủy thì chưa chắc B bị hủy .

- Là loại aggregation chặt chẽ hơn , còn gọi là non-shared aggregation
- Ký hiệu :



- Ví dụ :

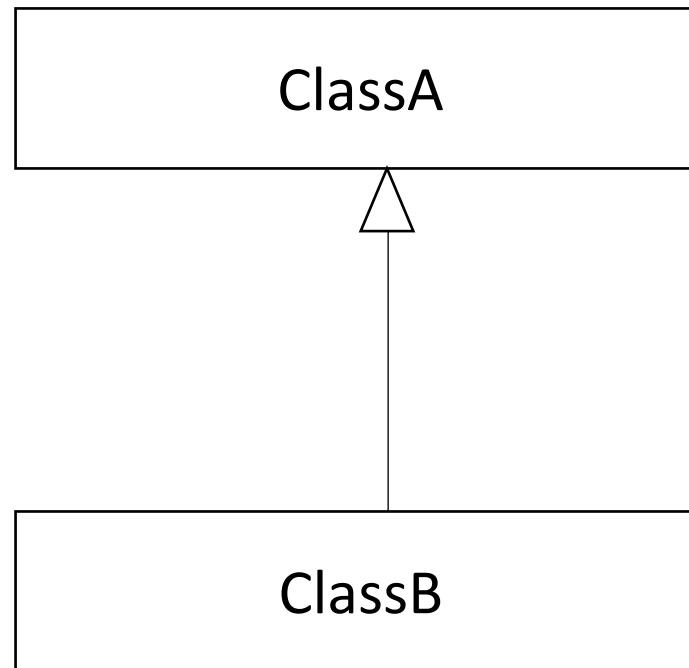


Quan hệ hợp thành

- Ý nghĩa : còn gọi là **Whole A – Part B** . Nghĩa là A được tạo từ nhiều B kết hợp lại , nhưng B không thể đứng 1 mình được , B chỉ là thuộc A mà thôi không thể cùng thuộc Whole khác được.
- Đã xác định được ClassA và ClassB có quan hệ Association với nhau
- Xác định rõ hơn:
 - Trong object của ClassA có chứa (trong phần thuộc tính) object của ClassB
 - ObjectX của ClassA bị hủy thì ObjectY của ClassB (bên trong ObjectX) không thể còn tồn tại
- Chú ý :
 - B *chỉ có thể là bộ phận* của whole A
 - A chết thì tất cả B chết
 - B chết không ảnh hưởng đến A
 - Bản số của Whole A luôn là 1, nghĩa là B luôn thuộc 1 A thôi

Đối tượng cụ thể (concrete) sẽ kế thừa các thuộc tính và phương thức của đối tượng tổng quát (general)

*Ký hiệu: **A is-a B***

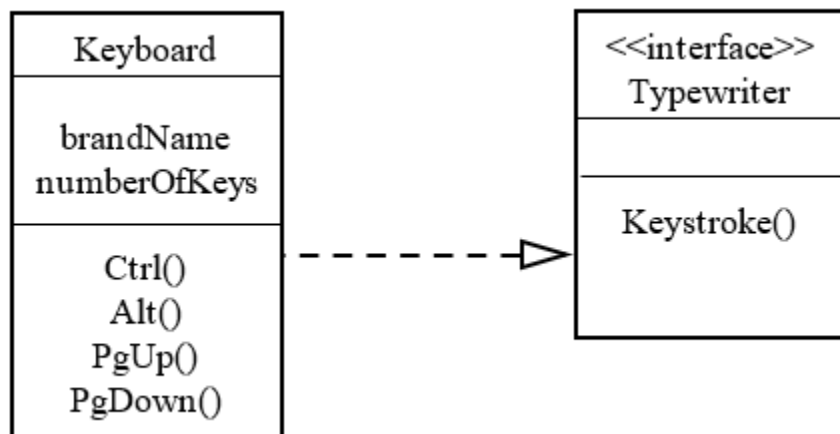


- A là tổng quát của B, B là chi tiết của A
- B là trường hợp đặc biệt của A
- A là cha của B, B là con của A

Là quan hệ giữa một classifier đóng vai trò là hợp đồng và một classifier đóng vai trò thực hiện. Hay nói cách khác:

*Mỗi quan hệ giữa 1 class implement 1 interface được gọi là quan hệ **realization**, được biểu diễn bởi đường đứt nét có hình mũi tên tam giác chỉ vào interface.*

Ký hiệu : có 2 loại ký hiệu



Hoặc



BÀI TẬP THỰC HÀNH

Bài tập 1

Cho các cặp lớp:

1. LopHoc – SinhVien
2. DonHang – ChiTietDonHang
3. Xe – DongCo
4. Nguoi – SinhVien

Yêu cầu

- Xác định loại quan hệ
- Giải thích lý do

Bài tập 2:

Cho hệ thống **Quản lý bán hàng**:

- Customer
- Order
- OrderDetail
- Product

Yêu cầu

1. Xác định mối quan hệ giữa các lớp
2. Chỉ ra:
 - Bội số (1-n, 1-1...)
3. Vẽ Class Diagram

BÀI GIẢI

BÀI 1

Cặp 1: LopHoc – SinhVien

Loại quan hệ

- **Association (kết hợp)**
 - Bội số: **1 – n**

Giải thích

- Một **LopHoc** có thể có **nhiều SinhVien**
- Một **SinhVien** thuộc về **một LopHoc**
- Hai đối tượng **tồn tại độc lập** với nhau
→ Sinh viên vẫn tồn tại dù lớp học bị xóa
- ✓ **Kết luận** Quan hệ giữa LopHoc và SinhVien là quan hệ Association với bội số 1–n.

Cặp 2: DonHang – ChiTietDonHang

Loại quan hệ

- **Composition (kết hợp chặt)**
 - Bội số: **1 – n**

Giải thích

- Một **DonHang** gồm nhiều **ChiTietDonHang**
- **ChiTietDonHang** không thể tồn tại độc lập
- Khi **DonHang** bị xóa → **ChiTietDonHang** cũng bị xóa

Kết luận

DonHang và **ChiTietDonHang** có quan hệ Composition vì vòng đời của **ChiTietDonHang** phụ thuộc vào **DonHang**.

Cặp 3: Xe – DongCo

Loại quan hệ

- Aggregation (kết hợp yếu)

Giải thích

- Xe có DongCo
- Nhưng DongCo có thể tồn tại độc lập
 - Có thể tháo động cơ
 - Có thể thay động cơ khác
- Vòng đời không phụ thuộc hoàn toàn

Kết luận

Quan hệ giữa Xe và DongCo là Aggregation vì DongCo có thể tồn tại độc lập với Xe.

Cặp 4: Nguoi – SinhVien

Loại quan hệ

- Inheritance (kế thừa)

Giải thích

- SinhVien là một Nguoi
- Sinh viên thừa hưởng:
 - Họ tên
 - Tuổi
 - Giới tính
- Và có thêm thuộc tính riêng:
 - Mã sinh viên
 - Điểm

Kết luận: SinhVien kế thừa từ Nguoi nên quan hệ giữa hai lớp là Inheritance.